UNIVERSIDADE FEDERAL DE UBERLÂNDIA

Matheus José

Desenvolvimento de um Sistema de Dispensa de Disciplinas

Uberlândia, Brasil

2022

UNIVERSIDADE FEDERAL DE UBERLÂNDIA

Matheus José

Desenvolvimento de um Sistema de Dispensa de Disciplinas

Trabalho de conclusão de curso apresentado à Faculdade de Computação da Universidade Federal de Uberlândia, como parte dos requisitos exigidos para a obtenção título de Bacharel em Ciência da Computação.

Orientador: Renato Pimentel

Universidade Federal de Uberlândia – UFU

Faculdade de Computação

Bacharelado em Ciência da Computação

Uberlândia, Brasil

2022

Matheus José

Desenvolvimento de um Sistema de Dispensa de Disciplinas

Trabalho de conclusão de curso apresentado à Faculdade de Computação da Universidade Federal de Uberlândia, como parte dos requisitos exigidos para a obtenção título de Bacharel em Ciência da Computação.

Trabalho em andamento. Uberlândia, Brasil, 22 de junho de 2022:

Renato Pimentel
Renato Pimentel

Professor

Professor

Uberlândia, Brasil
2022

Lista de ilustrações

Figura 1 – Diagrama de Sequência de Cadastro de Disciplinas 14

Lista de tabelas

Tabela 1 – Cronograma 9

Sumário

1 INTRODUÇÃO 6
1.1 Justificativa 7
1.2 Objetivos 7
1.3 Metodologia 8
1.4 Cronograma 9

2 REFERENCIAL TEÓRICO 10
2.1 Tecnologias 10
2.1.1 Java 10
2.1.2 Spring 10
2.1.3 Spring Boot 10
2.1.4 Spring Data 11
2.1.5 MongoDB 11
2.1.6 REST APIs 11
2.1.7 Microsserviços 12
2.1.8 Angular 12
2.1.9 NodeJS 12
2.1.10 Docker 12
2.1.11 Heroku 13
2.2 Modelagem de dados 13
2.3 Casos de uso 13

REFERÊNCIAS 15

1 Introdução

Com a crescente entrada de pessoas à universidade, as coordenações de cursos têm de lidar com as mais diversas situações da vida escolar dos alunos. Pode-se citar, por exemplo, jovens iniciando em determinado curso, porém que não se sentem tão confortáveis e acabam migrando para outro curso ou até mesmo para outra instituição de ensino, que vem a preencher vagas ociosas da instituição (VAGASOCIOSAS, 2022), e que abrem porventura solicitação para a coordenação para avaliar equivalência de disciplinas.

Essas solicitações que alunos encaminham às coordenações acabam gerando bastante trabalho, justamente porque existem inúmeros cenários distintos na análise de equivalência.

Hoje, na Universidade Federal de Uberlândia (UFU), quando um aluno requisita equivalência de disciplinas para a coordenação de curso, A análise é atribuída a um professor membro do colegiado do curso, que não necessariamente tem conhecimento da ementa ou experiência com determinadas disciplinas. O mesmo então compara a ementa que o aluno trouxe – do curso em que ele concluiu aquela matéria – com a do curso que ele está atualmente cursando na UFU. Essa parte é a mais minuciosa, pois requer uma análise crítica de cada componente da ementa, com base nas normas (NORMASEQUIVALENCIA, 2022). E então é dado um parecer da análise. Se aprovado, é emitida uma certidão comprobatória da equivalência. Caso não seja aceito, o aluno então poderá recorrer em um prazo estipulado.

Essa fase de análise de ementa é a que mais gasta esforço humano, de acordo com o processo atualmente existente de análise de equivalência, que é constituído das seguintes etapas: Primeiro o aluno abre uma solicitação à coordenação, e então membros do colegiado avaliam o pedido. Geralmente há um cabeça que é quem tem mais conhecimento sobre o assunto tratado na ementa da disciplina que foi requisitado equivalência. Nesse momento, caso algum outro aluno já tenha feito esse processo anteriormente e conseguiu equivalência, pode ser que o professor lembre desse fato e já aplique a equivalência. Caso contrário, então é analisado se a carga horária entre as disciplinas bate, além também dos conteúdos da ementa, e essa análise é feito conteúdo a conteúdo. Há casos em que os conteúdos entre as ementas divergem em relação a nomes, e então quem está analisando precisa ter mais cautela. Em caso de sucesso, e que dê equivalência, então posteriormente é emitido uma espécie de certidão comprovando que o aluno conseguiu equivalência. Caso não dê equivalência, e essa seja porque na ementa que o aluno mandou – do curso de origem – falte algum conteúdo que precise constar na nova ementa do curso que quer equivalência, então o professor pode pedir que o aluno faça uma prova e comprove que

possui conhecimento naquele certo assunto. Em outros casos, o aluno também pode recorrer em um prazo estipulado pela instituição. Dessa forma, como a demanda por solicitações de equivalência de disciplinas é muita alta, surgiu a ideia de implementar um sistema que consiga analisar se, entre duas ementas de disciplinas, essas sendo da mesma universidade ou não, possuem equivalência.

1.1 Justificativa

De acordo com a quantidade de vagas ociosas (VAGASOCIOSAS, 2022) na Universidade Federal de Uberlândia, pode-se perceber que muitos estudantes entram na universidade por transferência. E por entrarem por transferência, possivelmente será requisitado equivalência de disciplinas com base nas matérias que já foram concluídas na faculdade de origem. Com isso, para cada pedido de equivalência é aberto na coordenação uma solicitação, em que é analisada por um professor que tem conhecimento da ementa daquela disciplina. Com isso, é preciso bastante atenção e tempo para fazer a análise. Além de que caso o pedido seja indeferido, o aluno pode entrar com recurso para re-analisar. Dessa forma, o Sistema de Disciplina de Disciplinas é bastante útil, pois com ele é possível cadastrar ementas de disciplinas e validar se entre duas disciplinas elas são equivalentes, levando em consideração curso de origem e de destino. E com esse sistema também será gravado em uma base de dados as disciplinas que já tiveram equivalência, podendo assim consultar num futuro disciplinas que já foram analisadas, e consequentemente aumentando a agilidade para validar um processo de dispensa de disciplinas.

1.2 Objetivos

Desenvolver um sistema web para cadastrar disciplinas de outros cursos seja da UFU ou outras instituições, além de cadastrar sua equivalência com os cursos da Faculdade de Computação (FACOM). Isso permitirá agilizar o processo de avaliação e evitar decisões conflituosas entre processos de diferentes alunos.

1. Criar um *backend* para inserir dados na base de dados as ementas de disciplinas e seus respectivos cursos e universidades;
2. Criar um *frontend* para cadastro, listagem, deleção e atualização dessas disciplinas;
3. Implementar funcionalidade para receber uma ementa de disciplina e sua respectiva faculdade e comparar com outra ementa e ver se são equivalentes;
4. Criar funcionalidade para inserir na base de dados lendo de um arquivo PDF;
5. Criar funcionalidade para inserir na base de dados lendo de uma imagem (OCR);

6. Cadastrar processos de equivalência de alunos que já foram analisados.

1.3 Metodologia

Para o desenvolvimento do sistema de Dispensa de Disciplinas, será inicialmente feita a modelagem de dados do sistema. Essa modelagem será pensada primeiro nas regras de negócio conforme as normas de equivalência de disciplinas de graduação (NORMASEQUIVALENCIA, 2022).

Após a modelagem e diagramação do sistema e dos casos de uso, será utilizada a ferramenta de banco de dados não relacional MongoDB para persistência dos dados de disciplinas e de equivalências.

A linguagem Java com Spring, Spring Boot e Spring Data será utilizada para compor o *backend*, ou seja, as funcionalidades do sistema, como: cadastro de disciplinas e validação se duas disciplinas são equivalentes.

Além disso, com a modelagem do sistema e casos de uso, será montado algumas telas, como interface para login de usuário, uma página para cadastro das disciplinas e *upload* de PDF com as ementas e também uma tela para validar se duas disciplinas são equivalentes. Para isso será usado o *framework* Angular no desenvolvimento do *frontend*.

O sistema funcionará com base na arquitetura de microsserviços, ou seja, o *frontend* será separado do *backend*; ambos se integrarão via comunicação REST.

Como são duas partes separadas, *frontend* e *backend*, para desenvolvimento em ambiente local será utilizado um Dockerfile para automatizar o processo de subida do projeto.

Com o Dockerfile criado esse auxiliará na subida do projeto para produção, em que será feito *deploy* no Heroku.

Além de que todo o sistema será validado com testes unitários e funcionais, garantindo assim a qualidade do código, para não subir para ambiente produtivo com problemas de regra de negócio ou mesmo débitos técnicos.

Dado a aplicação em produção, então será disponibilizado um tutorial de como navegar no sistema, sobre a inserção de disciplinas e até mesmo de como usar a funcionalidade de validação de equivalências.

1.4 Cronograma

	Maio	Jun	Jul	Ago	Set	Out	Nov	Dez	Jan/23	Fev/23
Acompanhamento de disciplina PG1	X	X	X	X						
Levantamento de requisitos		X	X							
Definição de tecnologias		X								
Desenvolvimento banco de dados			X							
Backend: Desenvolvimento cadastro de disciplinas			X	X	X					
Backend: Avaliar equivalência entre duas disciplinas			X	X	X					
Backend: Cadastro de equivalências				X	X					
Backend: Desenvolvimento de cadastro de disciplinas por leitura de PDF/imagem					X	X				
Frontend: Desenvolvimento da tela de cadastro					X	X	X			
Frontend: Desenvolvimento da tela com resultados de equivalência					X	X	X			
Testes unitários / Funcionais				X	X	X				
Escrita texto	X	X	X	X	X	X	X	X		
Revisão texto							X	X	X	
Defesa										X

Tabela 1 – Cronograma

2 Referencial Teórico

Neste capítulo será tratado sobre a arquitetura do sistema, descrevendo as tecnologias utilizadas e modelagem de dados do sistema. Além de uma breve ilustração de casos de uso em que usuários operem as funcionalidades da aplicação.

2.1 Tecnologias

Nesta seção será descrito quais são as tecnologias escolhidas para este projeto, além de uma breve explicação sobre cada uma e sua importância no Sistema de Dispensa de Disciplinas.

2.1.1 Java

É uma linguagem de programação orientada a objetos comumente utilizada em grandes projetos. Com Java (JAVA, 2022) é possível a criação de aplicações multiplataforma, ou seja, é possível desenvolver para os seguintes sistemas operacionais: MAC, Linux, Windows; sem precisar preocupar com a arquitetura da máquina a qual está executando a aplicação. Além de que com Java dá pra criar aplicações WEB (servindo como *server-side*) e também *Desktop*.

2.1.2 Spring

O Spring (SPRING, 2022) foi criado por Rod Johnson para aumentar a agilidade no desenvolvimento de software. Justamente porque ele traz muitas funcionalidades de forma abstraída. Um exemplo é a criação de uma interface com Java Swing. Antes precisava-se validar se o cliente clicou no botão, já com o Spring basta implementar a ação para o botão, sem preocupar com o que acontecerá adiante. Além do mais temos injeções de dependência, em que uma dada Classe se preocupa apenas em utilizar os recursos que a dependência traz ao projeto. E o interessante é que dessa forma não é necessário ficar instanciado os objetos que são gerenciados pelo Spring.

2.1.3 Spring Boot

Assim como o Spring, o Spring Boot (SPRINGBOOT, 2022) vem para facilitar o desenvolvimento de aplicações Java. Justamente porque este é um *framework* que tem como objetivo facilitar a configuração inicial de um projeto. Com ele abstraímos diversas configurações iniciais, como: gerenciamento de dependências, bibliotecas, validar a saúde da aplicação (*health check*) e afins.

2.1.4 Spring Data

O Spring Data serve como um modelo dentro do Spring para acessar e manipular dados. Com ele conseguimos fazer uma conexão de forma mais fácil com bases de dados relacionais e não relacionais, além de que conseguimos implementar consultas, persistência de dados, atualização e deleção de dados de forma fácil. No caso deste projeto iremos utilizá-lo para manipular o banco de dados MongoDB. Segundo a (INFOQ, 2013), o Spring Data seria uma solução mais geral para persistência, justamente porque permite manipular os mais diversos bancos de dados.

2.1.5 MongoDB

É um banco de dados não relacional (*NoSQL*), orientado a Documentos. É muito utilizado quando é necessário ter uma rápida resposta de *input/output* de dados. Outro detalhe é que é mais recomendado em casos em que não é necessário um grande relacionamento de dados. A partir do (TECNOBLOG, 2022), é possível extrair algumas vantagens e desvantagens ao se trabalhar com MongoDB. As vantagens de utilizar um banco de dados não relacional é que tem-se uma grande flexibilidade para montar os documentos, além de não precisar montar o esquema de banco de dados, basta que seja pensado nas entidades do sistema, e também é possível aninhar dados e fazer as famosas agregações com eles, o que garante mais eficiência em buscas, sem precisar consultar as mais diversas coleções. Como desvantagem, fica o seu limite de dados por documentos.

2.1.6 REST APIs

Uma *Application Programming Interface Representational State Transfer* (API REST) pode ser dada como um guia de boas práticas que dita como APIs devem ser desenvolvidas, para que as aplicações de comuniquem devidamente. Essa arquitetura tem como alicerce o protocolo HTTP, e possui várias representações de recursos, como XML e JSON. Em uma API REST os métodos mais utilizado para efetuar operações no sistema é o GET, POST, PUT, DELETE ou PATCH. Definido o método HTTP a ser utilizado basta definir se terá um *body, path parameters* e *headers*. Requisitado ao sistema esse retorna um código indicando se deu sucesso ou não com a solicitação. Exemplos de código, são: código 200 quer dizer que deu tudo certo; 201 quer dizer quem um dado objeto foi criado com sucesso; 400 que foi uma requisição com falha; 404 que um objeto não foi encontrado; e 500 é uma mensagem que indica que teve um erro dentro do sistema, geralmente pode ser causado por exceção do tipo *null-pointer* e afins. Além disso, segundo (HOSTINGER, 2022) uma REST API é um conjunto de princípios de arquitetura de software, já uma API RESTful é a implementação das práticas do REST em uma API, seguindo suas conformidades. Dessa forma, este projeto, por exemplo, seguirá os princípios do REST e implementará APIs RESTful.

2.1.7 Microsserviços

Segundo a Amazon Web Service (AWS), microsserviços consistem em pequenos serviços que são independentes e que se comunicam por APIs bem definidas para gerar o sistema final (MICROSSERVICOS, 2022). Essa abordagem de desenvolvimento de sistemas é interessante pois o sistema poderá ser criado com base na necessidade, ou seja, em caso de necessidade de desenvolver parte do sistema em *Java*, *Python* ou qualquer linguagem que seja, essas poderão se comunicar desde que obedeçam um contrato da definição das APIs. Outra vantagem é na manutenção do código, pois ao criar pequenos serviços as classes de domínio ficam com a lógica reservada a aquele escopo, o que fará com que um outro microsserviço não implique diretamente, com isso se precisar fazer qualquer manutenção que seja será mais simples e possivelmente quebrará menos coisas no sistema.

2.1.8 Angular

O Angular (ANGULAR, 2022) é um *framework* que utiliza TypeScript e HTML para efetuar a criação da interface com o usuário. Ou seja, é o *frontend* das aplicações web. Esse *framework* é muito utilizado para aprimorar o desenvolvimento de software na camada web, justamente porque com ele conseguimos ter maior consistência e capacidade para reutilizar códigos. Como é baseado no TypeScript, que é da Microsoft, temos uma maior organização do código além de um ambiente mais seguro.

2.1.9 NodeJS

O NodeJS (NODEJS, 2022) é um ambiente de execução Javascript, isso quer dizer que é possível criar aplicações Javascript que rodarão em uma máquina, ou seja, não dependerá do navegador para rodar. Para o caso deste projeto, o NodeJS é necessário para que criemos a *engine* da aplicação Angular no *frontend*. Do NodeJS será utilizado o gerenciador de pacotes, o npm. O npm é similar ao maven, que é muito usado em aplicações Java.

2.1.10 Docker

O *Docker* (DOCKER, 2022) é uma ferramenta que possibilita a criação e administração de ambientes isolados. Com ele é possível empacotar uma aplicação ou ambiente dentro de um *container*, e qualquer ambiente que tenha o Docker instalado conseguirá desempacotar e executar o que tem ali dentro. Por exemplo, é possível subir um banco de dados *NoSQL* MongoDB a partir de uma imagem Docker, e então utilizar isso em um sistema que ainda está sendo desenvolvido. A vantagem disso é que não é necessário contratar um banco de dados enquanto a aplicação ainda está em estágio inicial. Ou seja,

ganha-se velocidade no desenvolvimento. Além de que para fazer *build* e *deploy* da aplicação o Docker é uma ferramenta essencial, justamente por fazer tão bem o empacotamento da aplicação.

2.1.11 Heroku

O Heroku (HEROKU, 2022) é uma plataforma para hospedagem e testagem de aplicações virtuais na nuvem. Essa plataforma é muito utilizada pelos desenvolvedores para publicação de suas aplicações, justamente pela sua facilidade na configuração da infraestrutura para *deploy*. Ele será utilizado para publicar o sistema deste projeto, tanto a parte *backend* quanto *frontend*.

2.2 Modelagem de dados

Segundo a (INFONOVA, 2022), uma boa modelagem de dados permite com que uma dada organização chegue em seus objetivos aproveitando ao máximo os seus dados, conseguindo suprir todas as demandas de negócio. Sendo assim, um modelo de dados em suma compõe as regras de negócio do sistema. E é necessário uma boa modelagem de dados para que o sistema se molde e se perpetue no tempo sem muitas complicações. No caso deste sistema, optou-se pela modelagem de dados de Documentos, por conta do banco escolhido ser um banco não relacional (NoSQL). E assim definido os modelos, é possível começar o desenvolvimento sem muitas complicações, justamente por ter sido pensado o sistema a mais alto nível.

2.3 Casos de uso

Um caso de uso pode ser dito como uma funcionalidade que o sistema deve ter, um comportamento. Por exemplo, nesse sistema teremos um cadastro de disciplinas. A *API* que será criada para fazer esse cadastro pode ser dada como um caso de uso. Geralmente para se representar um caso de uso é utilizado Diagramas de Sequência para mostrar como se dará a jornada do usuário, desde a sua chamada no *frontend* até os passos que o *backend* fará, seja indo na base de dados ou até mesmo se integrando com outro sistema.

Como exemplo, pode-se ver na Figura 1 a ilustração de um diagrama de sequência para o caso de uso do cadastro de disciplinas no sistema de equivalência.

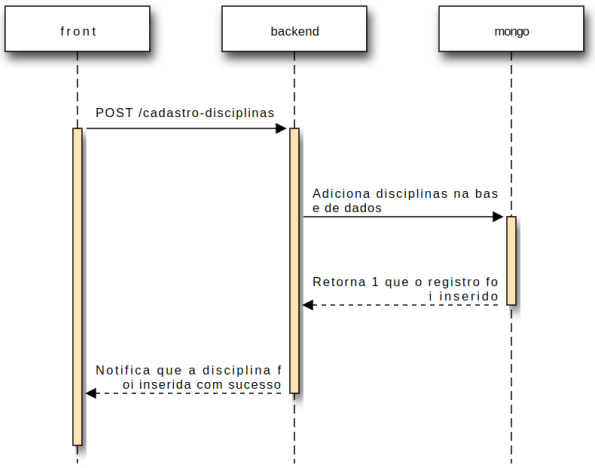


Figura 1 – Diagrama de Sequência de Cadastro de Disciplinas.

Referências

ANGULAR. **Angular, plataforma para desenvolvimento web**. 2022. Desenvolvimento web. Disponível em: <https://angular.io/>. Acesso em: 17 ago. 2022. Citado na página 12.

DOCKER. **O que é Docker?** 2022. Conteinerização. Disponível em: <https://stack.desenvolvedor.expert/appendix/docker/oquee.html>. Acesso em: 17 ago. 2022. Citado na página 12.

HEROKU. **Heroku, hospedagem de aplicações**. 2022. Gerenciamento de aplicações em nuvem. Disponível em: <https://www.heroku.com/>. Acesso em: 17 ago. 2022. Citado na página 13.

HOSTINGER. **O Que é uma API RESTful e Porque Isso Importa**. 2022. Princípios de arquitetura de software. Disponível em: <https://www.hostinger.com.br/tutoriais/api-restful>. Acesso em: 17 ago. 2022. Citado na página 11.

INFONOVA. **O que são modelos de dados?** 2022. O que são modelos de dados? Disponível em: <https://www.infonova.com.br/gestao-de-ti/o-que-sao-modelos-de-dados/>. Acesso em: 17 ago. 2022. Citado na página 13.

INFOQ. **Spring Data: A solução mais geral para persistência?** 2013. Maleabilidade ao manusear bancos de dados. Disponível em: <https://www.infoq.com/br/articles/spring-data-intro/>. Acesso em: 17 ago. 2022. Citado na página 11.

JAVA. **O que é tecnologia Java e por que preciso dela?** 2022. Desenvolvimento backend. Disponível em: <https://www.java.com/pt-BR/download/help/whatis_java.html>. Acesso em: 17 ago. 2022. Citado na página 10.

MICROSSERVICOS. **O que são microsserviços?** 2022. Sistemas distribuídos. Disponível em: <https://aws.amazon.com/pt/microservices/>. Acesso em: 17 ago. 2022. Citado na página 12.

NODEJS. **NodeJS**. 2022. Execução de código JavaScript fora de nevegador web. Disponível em: <https://nodejs.org/en/>. Acesso em: 17 ago. 2022. Citado na página 12.

NORMASEQUIVALENCIA. **Resolução Congrad Nº 46**. 2022. Normas equivalência de disciplinas. Disponível em: <http://www.reitoria.ufu.br/Resolucoes/resolucaoCONGRAD-2022-46.pdf>. Acesso em: 27 jun. 2022. Citado 2 vezes nas páginas 6 e 8.

SPRING. **Documentação Spring Framework**. 2022. Documentação Spring Framework. Disponível em: <https://docs.spring.io/spring-framework/docs/current/reference/html/>. Acesso em: 20 jul. 2022. Citado na página 10.

SPRINGBOOT. **Spring Boot, maior facilidade em criar aplicações**. 2022. Agilidade ao criar aplicações. Disponível em: <https://spring.io/projects/spring-boot>. Acesso em: 17 ago. 2022. Citado na página 10.

TECNOBLOG. **O que é e para que serve o MongoDB?** 2022. Banco de Dados Não Relacional. Disponível em: <https://tecnoblog.net/responde/o-que-e-e-para-que-serve-o-mongodb/>. Acesso em: 17 ago. 2022. Citado na página 11.

VAGASOCIOSAS. **Transferências para a UFU para preenchimento de vagas ociosas**. 2022. UFU tem inscrições abertas para preenchimento de quase 2 mil vagas ociosas. Disponível em: <https://g1.globo.com/mg/triangulo-mineiro/noticia/2022/03/24/ufu-tem-inscricoes-abertas-para-preenchimento-de-quase-2-mil-vagas-ociosas.ghtml>. Acesso em: 27 jun. 2022. Citado 2 vezes nas páginas 6 e 7.